Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251218**
Nama Lengkap	**Agatha Sabda Alethea Prabawa**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Fungsi, Parameter, dan Argumen

Fungsi (*function*) adalah blok kode yang dirancang untuk melakukan tugas tertentu dan fungsi diberi nama agar dapat dipanggil berkali-kali (skodev, 2023). Sebuah fungsi dapat menerima *input* yang dikenal sebagai parameter, dan setelah melakukan serangkaian instruksi, akan menghasilkan *output*. Fungsi memiliki beberapa jenis sebagai berikut:

1. Fungsi bawaan, yaitu fungsi-fungsi yang sudah disediakan oleh Bahasa pemrograman itu sendiri. Contohnya dalam python yaitu input() yang biasa digunakan untuk menerima input dan print() yang digunakan untuk menampilkan output ke layar.

```
inputSuhu = input("masukan suhu tubuh: ")
    print("suhu harus berupa angka!")
```

2. Fungsi pengguna, fungsi yang dibuat oleh programmer sesuai dengan kebutuhan program, kita dapat mendefinisikan fungsi sendiri untuk menjalankan tugas-tugas tertentu. Contoh sebagai berikut:

```
def programBMI():
    try:
        print("Program Menghitung BMI")
```

3. Fungsi anonim atau *lambda function*, fungsi tanpa nama ini sering kali digunakan untuk operasi sederhana dan digunakan hanya satu kali dalam kode. Contoh sebagai berikut:

```
97    kuadratLambda = lambda x: x ** 2
98
99    print(kuadratLambda(6))
100   print(kuadratLambda(7))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    POI

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> &
s/python3.11.exe "d:/KULIAH/MATA KULIAH SEMESTEF
49
25
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> &
s/python3.11.exe "d:/KULIAH/MATA KULIAH SEMESTEF
36
49
```

4. Fungsi rekursif, yaitu fungsi yang memanggil dirinya sendiri ketika program tidak berhenti di kondisi pertama yang diberi. Contoh sebagai berikut:

```
102   def faktorial(x):
103       if x <= 1:
104           return 1
105       else:
106           return x * faktorial(x-1)
107
108   print(faktorial(7))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    POR

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> &
s/python3.11.exe "d:/KULIAH/MATA KULIAH SEMESTER
5040
```

Ketika kondisi nilai x tidak kurang dari sama dengan 1, maka program akan memanggil dirinya sendiri dengan (x-1) sampai nilai x bernilai kurang dari sama dengan 1 dan hasilnya akan dikalikan dengan x.

Pada contoh rekursif, terdapat beberapa komponen lain yaitu parameter dan argumen. Parameter adalah sebutan untuk nilai inputan fungsi pada saat fungsi itu di defnisikan, sementara argumen adalah sebutan untuk nilai inputan fungsi pada saat fungsi itu dipanggil. Pada contoh

rekursif diatas, ‘x’ didalam faktorial(x) merupakan sebuah parameter dan ‘7’ pada print(faktorial(7)) merupakan argumennya.

Return Value

Return value adalah nilai yang dikembalikan oleh fungsi kepada pemanggilnya ketika fungsi selesai dieksekusi. Fungsi dapat mengirimkan hasil komputasinya kembali ke bagian program yang memanggilnya menggunakan kata kunci *return.* Beberapa karakteristik dari *return* yaitu, berfungsi untuk mengakhiri eksekusi fungsi sehingga kode setelah *return* tidak akan dijalankan, digunakan untuk mengembalikan nilai ke pemanggil, dalam satu fungsi bisa ada lebih dari satu *return*, jika suatu fungsi tidak ada *return* maka fungsi mengembalikan nilai *none*.

```
110 def test(message):
111     print(message)
112     print(message)
113
114 print(test("hola semua"))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:
hon3.11.exe "d:/KULIAH/MATA KULIAH SEMESTER 2/PrakA
hola semua
hola semua
None
```

Pada contoh gambar diatas output yang dihasilkan “hola semua” sebanyak dua kali didapatkan dari print(message) dan *none* muncul karena fungsi ini tidak memiliki return untuk mengembalikan nilai. Agar program ini tidak menghasilkan nilai *none*, maka kita perlu menambahkan *return* nilai agar isi dari parameter message ditampilkan. Contoh pada gambar berikut ini.

```
110   def test(message):
111       print(message)
112       print(message)
113       return message
114
115   print(test("hola semua"))
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMIN

PS D:\KULIAH\MATA KULIAH SEMESTER 2\Prak
hon3.11.exe "d:/KULIAH/MATA KULIAH SEMES
hola semua
hola semua
hola semua
```

Optional Argument dan Named Argument

Pada python, kita dapat memanggil fungsi dengan berbagai cara. Dalam fungsi terdapat 2 jenis parameter, yaitu *optional argument* (default parameter) dan *named argument* (keyword argument).

1. Optional Argument adalah parameter yang memiliki nilai default. Jika user tidak mengisikan nilai argumen saat memanggil fungsi, maka nilai defaultlah yang akan ditampilkan/digunakan. Contoh sebagai berikut:

```
126 def salam(nama, sapaan="hai"):
127     print(f"{sapaan}, {nama}")
128
129 salam("gatha", "selamat pagi")
130 salam("gatha")
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    P

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro>
hon3.11.exe "d:/KULIAH/MATA KULIAH SEMESTER 2/
selamat pagi, gatha
hai, gatha
```

Ketika user mengisikan argument dari kedua parameter yang ada (nama dan sapaan), maka output yang dihasilkan adalah “selamat pagi, gatha”. Namun, jika user hanya

mengisikan parameter nama, maka program akan menggunakan argumen default yaitu sapaan="hai" dan output yang dihasilkan adalah "hai, gatha".

2. Named argument adalah cara memanggil fungsi dengan menyebutkan nama parameter secara eksplisit, sehingga urutan argument tidak begitu diperhatikan. Contoh sebagai berikut:

```
132  def identitas(nama, umur, asal):
133      print(f"nama: {nama}")
134      print(f"umur: {umur}")
135      print(f"asal: {asal}")
136
137  identitas("gatha", 20, "kediri")
138  # identitas(umur=20, nama="gatha"
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINA

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakA
1.exe "d:/KULIAH/MATA KULIAH SEMESTER 2/Pr
nama: gatha
umur: 20
asal: kediri
```

Gambar diatas merupakan contoh ketika parameter diisi secara berurutan.

```
132  def identitas(nama, umur, asal):
133      print(f"nama: {nama}")
134      print(f"umur: {umur}")
135      print(f"asal: {asal}")
136
137  # identitas("gatha", 20, "kediri")
138  identitas(umur=20, nama="gatha", asal="kediri")
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/
1.exe "d:/KULIAH/MATA KULIAH SEMESTER 2/PrakAlPro/testing.
nama: gatha
umur: 20
asal: kediri
```

Gambar diatas ini merupakan contoh ketika parameter tidak diisi secara berurutan. Jika dibandingkan, hasilnya sama saja.

Anonymus Function (Lambda)

Anoymus Function atau yang sering disebut Lambda adalah fungsi yang tidak memiliki nama. Lambda biasanya disimpan ke dalam suatu variabel atau dieksekusi langsung. Seperti fungsi pada umumnya, Lambda juga bisa memiliki parameter dan mengembalikan nilai. Yang membedakan lambda dan fungsi adalah lambda hanya boleh diisi satu baris statement. Karena pada dasarnya lambda hanya biasa digunakan untuk statement sederhana, jika ingin membuat program yang membutuhkan statement lebih panjang maka direkomendasikan untuk menggunakan fungsi saja. Contoh penggunaan fungsi lambda seperti yang ada pada subab materi fungsi, parameter, dan argumen. Berikut contoh tambahan dari penggunaan lambda pada python jika memiliki dua parameter.

```
140   nilaiMaks = lambda x, y: x if x > y else y
141
142   print(f"angka terbesar dari 6 dan 7: {nilaiMaks(7, 6)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/lenovo/AppDat
AH SEMESTER 2/PrakAlPro/testing.py"
angka terbesar dari 6 dan 7: 7
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: https://github.com/gathasja/71251218_Agatha.git

SOAL 1

soal01.py | soal02.py | soal03.py

PrakAlPro-71251218 > minggu05 > soal01.py > ...

```python
def cek_angka(x, y, z):
    if x == y or x == z or y == z:
        return False
    else:
        return (x + y == z) or (x + z == y) or (y + z == x)

print(cek_angka(6, 7, 8))
print(cek_angka(6, 7, 13))
print(cek_angka(6, 6, 7))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro> & C:/Users/lenovo/AppDat
TA KULIAH SEMESTER 2/PrakAlPro/PrakAlPro-71251218/minggu05/soal01.py"
False
True
False
```

Penjelasan langkah perbaris dari soal01:

1. def cek_angka(x, y, z): -> mendefinisikan fungsi Bernama cek_angka() yang menerima 3 parameter x, y, dan z. Nantinya fungsi ini akan mengembalikan nilai.
2. if x == y or x == z or y == z:

   return False -> untuk memeriksa apakah ada 2 angka yang bernilai sama? jika minimal ada 1 pasangan yang sama, maka kondisi akan bernilai *false*.
3. else:

   return (x + y == z) or (x + z == y) or (y + z == x) -> line ini hanya akan dijalankan ketika kondisi awal yang diminta tidak terpenuhi. Di bagian ini akan di periksa apakah ada kemungkinan 2 angka jika dijumlahkan hasilnya sama dengan angka ke-3? Jika ya, maka fungsi akan menghasilkan nilai True dan jika tidak, maka fungsi akan menghasilkan nilai False.

4. print(cek_angka(6, 7, 8)) -> memanggil fungsi dengan nilai 6, 7, 8 dan menampilkan hasilnya (False).

   print(cek_angka(6, 7, 13)) -> memanggil fungsi dengan nilai 6, 7, 13 dan menampilkan hasilnya (True).

   print(cek_angka(6, 6, 7)) -> memanggil fungsi dengan nilai 6, 6, 8 dan menampilkan hasilnya (False).

SOAL 2

soal01.py | soal02.py | soal03.py

PrakAlPro-71251218 > minggu05 > soal02.py > ...

```python
def cek_digit_belakang(x, y, z):
    return (x % 10 == y % 10) or (x % 10 == z % 10) or (y % 10 == z % 10)

try:
    x = int(input("nilai x: "))
    y = int(input("nilai y: "))
    z = int(input("nilai z: "))

    hasil = cek_digit_belakang(x, y, z)
    print(hasil)

except ValueError:
        print ("input harus berupa angka!")
```

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro
02.py"
nilai x: 30
nilai y: 20
nilai z: 18
True
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro
02.py"
nilai x: 145
nilai y: 5
nilai z: 100
True
PS D:\KULIAH\MATA KULIAH SEMESTER 2\PrakAlPro
02.py"
nilai x: 71
nilai y: 187
nilai z: 18
False
```

```
PS D:\KULIAH\MATA KULIAH SEMESTER 2\
02.py"
nilai x: 1024
nilai y: 14
nilai z: 94
True
PS D:\KULIAH\MATA KULIAH SEMESTER 2\
02.py"
nilai x: 53
nilai y: 8900
nilai z: 658
False
```

Penjelasan langkah perbaris dari soal02:

1. def cek_digit_belakang(x, y, z): -> mendefinisikan fungsi Bernama cek_digit_belakang() yang menerima 3 parameter x, y, dan z dan akan mengembalikan nilai berdasarkan pengecekan digit terakhir.
2. return (x % 10 == y % 10) or (x % 10 == z % 10) or (y % 10 == z % 10) -> memeriksa apakah ada minimal 2 dari 3 angka memiliki digit terakhir/satuan yang sama dengan menggunakan operasi modulus, dan mengembalikan hasilnya.
3. try: -> digunakan untuk menyimpan kode-kode program yang akan dijalankan dan berpotensi menyebabkan kesalahan atau *error*.
4. x = int(input("nilai x: "))

   y = int(input("nilai y: "))

z = int(input("nilai z: ")) -> basically, ketiga baris ini memiliki kegunaan yang sama yaitu untuk meminta user mengisikan nilai parameter, mengkonversi ke integer, dan menyimpannya ke dalam variabel (x, y, z).

5. hasil = cek_digit_belakang(x, y, z) -> memanggil fungsi cek_digit_belakang dengan nilai x, y,z yang sudah diinputkan oleh user lalu menyimpan hasilnya di dalam variabel hasil.

   print(hasil) -> menampilkan hasil pengecekan.

6. except ValueError:

   print ("input harus berupa angka!") -> ‘except’ digunakan untuk menangani *error* yang terjadi dalam ‘try’ dan ‘ValueError’ merupakan jenis *error* yang akan ditampilkan ketika terjadi kesalahan pada program ini adalah ‘nilai harus berupa angka!’.

SOAL 3

soal01.py | soal02.py | soal03.py

PrakAlPro-71251218 > minggu05 > soal03.py > ...

```python
celciusToFahrenheit = lambda c : (9/5) * c + 32
celciusToReamur = lambda c : 0.8 * c

print(int(celciusToFahrenheit(100)))
print(int(celciusToReamur(80)))
print(int(celciusToFahrenheit(0)))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PrakAlPro> & C:/Users/lenovo/AppData/Local/Microsoft/Windc
PrakAlPro> & C:/Users/lenovo/AppData/Local/Microsoft/Windc
lPro-71251218/minggu05/soal03.py"
212
64
32
```

Penjelasan langkah perbaris dari soal03:

1. celciusToFahrenheit = lambda c : (9/5) * c + 32 -> membuat fungsi lambda untuk mengkonversi suhu dari celcius ke fahrenheit, fungsi ini menerima satu parameter (c) dan mengembalikan hasil perhitungan dari rumus konversi suhu $F = \left(\frac{9}{5}\right) * C + 32$.
2. celciusToReamur = lambda c : 0.8 * c -> membuat fungsi lambda untuk mengkonversi suhu dari celcius ke reamur, fungsi ini menerima satu parameter (c) dan mengembalikan hasil perhitungan dari rumus konversi suhu $R = 0.8 * C$.
3. print(int(celciusToFahrenheit(100))) -> menghitung konversi 100°C ke fahrenheit, membulatkan ke integer, lalu hasilnya ditampilkan.
4. print(int(celciusToReamur(80))) -> menghitung konversi 80°C ke reamur, membulatkan ke integer, lalu hasilnya ditampilkan.
5. print(int(celciusToFahrenheit(0))) -> menghitung konversi 0°C ke fahrenheit, membulatkan ke integer, lalu hasilnya ditampilkan.